SAVEETHA SCHOOL OF ENGINEERING

C PROGRAMMING ASSIGNMENT

Comprehensive System Design and Implementation Challenge

Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Comprehensive System Design and Implementation Challenge – District Blood Bank Network
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	13 NOVEMBER 2026

TASK A2 — Design with Recursion (5 Marks)

Task / Problem Statement

Donor IDs carry a check digit equal to the digital root of the other digits. The blood bank also computes base^n for a reporting formula. Design both recursively.

Aim

To express the digital root and fast exponentiation as recursive algorithms, trace them and analyse their cost and risks.

A2.1 – Recursive Design (2 marks)

```
ALGORITHM DigitSum(n)                ALGORITHM DigitalRoot(n)
  INPUT  : n >= 0                      INPUT  : n >= 0
  OUTPUT : sum of the digits of n      OUTPUT : single digit 0..9
BEGIN                                BEGIN
  IF n < 10 THEN                       IF n < 10 THEN
    RETURN n            (base)           RETURN n            (base)
  END IF                               END IF
  RETURN (n MOD 10) + DigitSum(n DIV 10)  RETURN DigitalRoot(DigitSum(n))   (recursive)
END                                  END
```

```
ALGORITHM Power(base, n)             ALGORITHM PowerLinear(base, n)
  INPUT  : base, n >= 0                INPUT  : base, n >= 0
  OUTPUT : base^n                      OUTPUT : base^n
BEGIN                                BEGIN
  IF n = 0 THEN                        IF n = 0 THEN
    RETURN 1            (base)           RETURN 1            (base)
  END IF                               END IF
  half <- Power(base, n DIV 2)  (ONCE)    RETURN base * PowerLinear(base, n - 1)
  IF n MOD 2 = 0 THEN                  END
    RETURN half * half
  ELSE
    RETURN half * half * base
  END IF
END
```

Function	Base case	Recursive case
DigitalRoot(n)	n < 10 → return n	DigitalRoot(DigitSum(n)) – the number gets smaller every time
DigitSum(n)	n < 10 → return n	(n mod 10) + DigitSum(n div 10) – one digit less each call
Power(base, n) [halving]	n = 0 → return 1	(Power(base, n div 2))², times base if n is odd – exponent halves
PowerLinear(base, n)	n = 0 → return 1	base × PowerLinear(base, n−1) – exponent drops by 1

A2.2 – Trace the Calls (2 marks)

Power(2, 10): Power(10) → Power(5) → Power(2) → Power(1) → Power(0) = 1. Returning: Power(1) = 1·1·2 = 2; Power(2) = 2^2 = 4; Power(5) = 4^2·2 = 32; Power(10) = 32^2 = **1024**. Five calls, depth 5.

DigitalRoot(9875): DigitSum(9875) = 5 + DigitSum(987) = 5 + 7 + DigitSum(98) = 5+7+8+DigitSum(9) = 5+7+8+9 = 29, so DigitalRoot(29) → DigitSum(29) = 11 → DigitalRoot(11) → DigitSum(11) = 2 → DigitalRoot(2) = **2** (check digit 2).

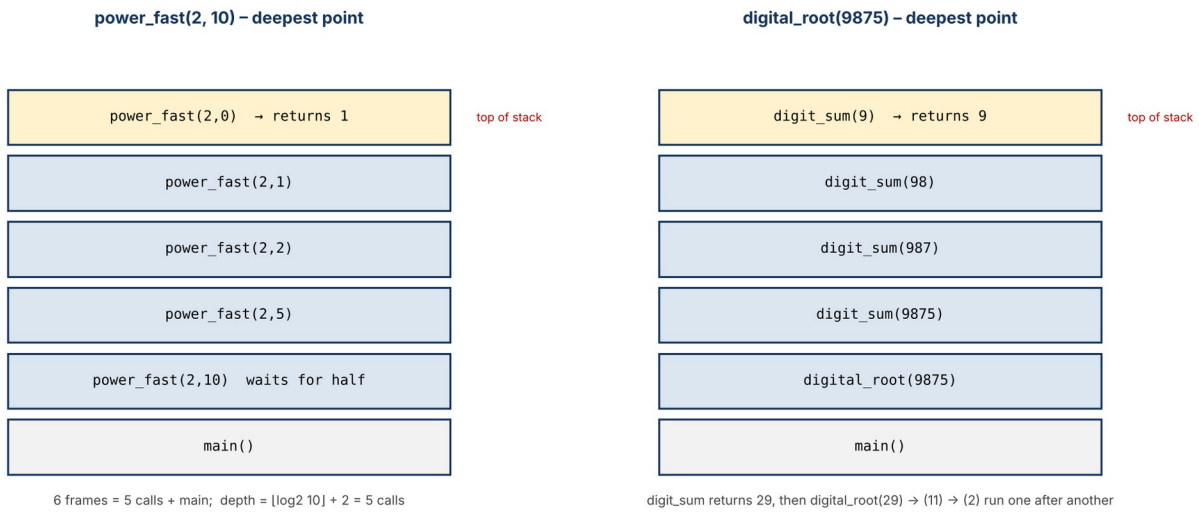


Figure A2 – Call stack at the deepest point of each trace (top of stack is the newest call).

Recurrences (T = number of calls):

Version	Recurrence	Solution	Value for n = 30
Linear	$T(0) = 1;\ T(n) = 1 + T(n-1)$	$T(n) = n + 1 \rightarrow O(n)$	31
Halving (one recursive call)	$T(0) = 1;\ T(n) = 1 + T(\lfloor n/2 \rfloor)$	$T(n) = \lfloor \log_2 n \rfloor + 2 \rightarrow O(\log n)$	6
Halving but two calls (the wrong way)	$T(0) = 1;\ T(n) = 1 + 2 \cdot T(\lfloor n/2 \rfloor)$	$T(n) = 2^{(\lfloor \log_2 n \rfloor + 2)} - 1 \rightarrow O(n)$	63
DigitSum with d digits	$T(1 \text{ digit}) = 1;\ T(d) = 1 + T(d-1)$	$T(d) = d$	–

A2.3 – Recursion Risks (1 mark)

- **Missing base case:** the function never stops calling itself. Every call adds a stack frame (return address, parameters, locals), the stack (about 8 MB) fills up and the program crashes with a **stack overflow** (segmentation fault). B1.4 shows this happening at n = 261,402.
- **Replace recursion with a loop when** the depth grows with the input size (linear recursion such as PowerLinear, or a list with 50,000 donors); when a loop is just as clear; when the function is called very often (call overhead); or on small-memory systems. Keep recursion when the depth is small and the problem is naturally recursive (halving, trees).

Alternative Concept and Complexity Analysis – loop or formula for the digital root

```
ALGORITHM DigitalRootLoop(n)            ALGORITHM DigitalRootFormula(n)
BEGIN                                   BEGIN
  WHILE n >= 10 DO                        IF n = 0 THEN RETURN 0
    s <- 0                              RETURN 1 + (n - 1) MOD 9
    WHILE n > 0 DO                      END
      s <- s + n MOD 10 ; n <- n DIV 10
    END WHILE
    n <- s
  END WHILE
  RETURN n
END
```

Version	Time	Extra stack space	Comment
Recursive (DigitSum depth d)	O(d), d = number of digits	O(d) frames	Clear, mirrors the definition

Loop	O(d) for the first pass; later passes work on at most 2-digit numbers	O(1)	No stack growth; works for any size
Formula 1 + (n−1) mod 9	**O(1)** – one division	O(1)	Fastest, but needs the maths insight; special-case n = 0

For a 64-bit number $d \le 19$, so all three are practically instant; the choice is about clarity and safety. B1 confirms that the formula and the recursive function agree on every test value.

Result

Recursive designs with base cases, traces, recurrences ($n + 1$, $\lfloor \log_2 n \rfloor + 2$, $\sim 2n$) and risks were produced; loop and formula alternatives were compared.